

Pentium processors. Processors since that time have been *out-of-order* processors (no, not in the “doesn’t work” sense), and in the world of general-purpose processing (office applications and so on), it’s these out-of-order processors that deliver a better performance. So why did IBM take Cell back into the Dark Ages? Let’s understand the difference between in-order and out-of-order execution first. Consider these instructions:

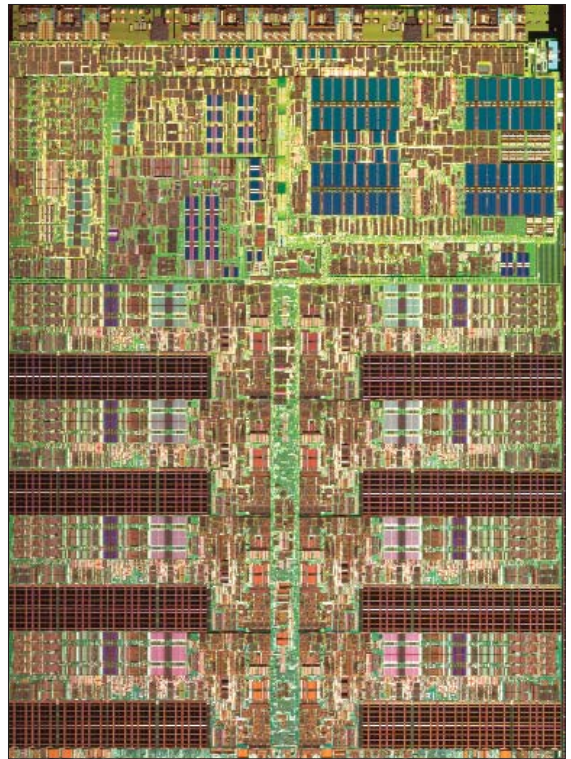
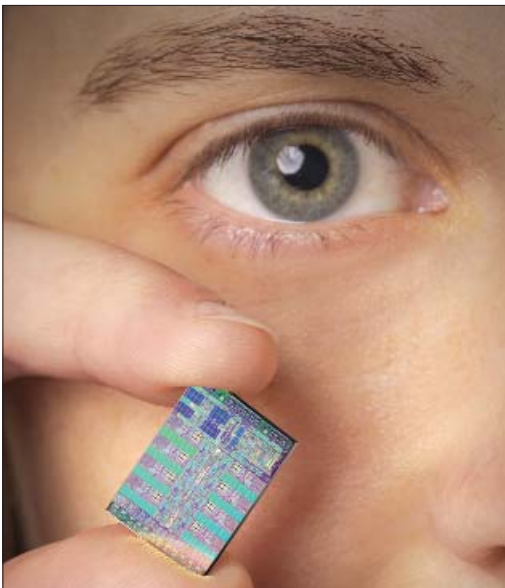
1. $A = B + C$
2. $D = A + E$
3. $X = Y + Z$

You’ll notice that instruction 2 has to be executed after 1, because it depends on the resulting value of A. Instruction 3, however, is independent of the other two. An in-order processor, as the name suggests, will execute these instructions in the order that it receives them. If all the data it needs is readily available in the processor’s cache, then the instructions proceed swiftly, and all is well.

Suppose, now, that the value of B isn’t in the cache, but C is. It’s only a matter of four or five CPU clock cycles for it to fetch C, but to get to B, it has to first search the cache and encounter a “cache miss,” following which it has to access the system’s main memory to get its data, resulting in a delay of a couple of hundred CPU cycles.

All this time, instruction 3 has to wait its turn, even though the idle CPU cycles which were wasted hunting down B could have been used to process it and get the job over with. This is where out-of-order processors came in. Out-of-order processors use an “Instruction Window” (something like a buffer where incoming instructions are stored), within which it looks for instructions that can be run independently, and processes them in parallel, so a cache miss isn’t such a disaster.

Put simply, the difference is the same as between shopping for items in the order that they’re written on your list, or by picking up whatever items on the list that you can see,



A microscopic view of the Cell—the black “bands” you see on either side are the SPEs; the PPE occupies the top left

irrespective of order. Out-of-order processors have been immensely successful in multi-tasking environments (which is the most common scenario for PC use) because they receive instructions from multiple programs, all independent of each other.

Coming back to our original question: why does the Cell forego such obvious advantages to go with the in-order approach?

Firstly, apart from the dire consequences of a cache miss, in-order cores perform quite well. Secondly, and more importantly, in-order processors are simple—without the added circuitry that enables out-of-order execution, the transistor count of the in-order core is low, which is how they’re able to fit nine cores on that little chip. To get around this cache-miss hassle, IBM has resorted to a neat trick.

We mentioned before that each SPE has its own local storage rather than a cache—this is because unlike cache memory, which has its caching logic hard-wired into it, this local store is accessible to the programmer. What this means is that the onus is now on the programmer (rather, the compiler) to ensure that any data that the SPE needs is available in this local store exactly when it needs it, minimising delays, or at the very least, making them predictable. Note that this applies only to the SPEs—the PPE still has a traditional L1 cache, and gets its performance from the speeds that the Cell will run at—between 3.2 and 4 GHz.

Raw speed isn’t the only thing behind the Cell, though. It does pack another really nifty feature: